Code review — booking

Rareș, runde de review pe proiectul tău, cea mai nouă sus.

Runda 2 — 2026-11-03 — sha 7d89902

Scop: `solved B1, B2` + `B3 also fixed. Code review complete`. Poarta de compilare: compilează, **337 din 337 de teste trec** (în runda 1: 321/322). Aplicația rulată pe MySQL 8.4 în Docker, profil `dev`, iar fiecare caz de mai jos e trimis prin `curl`, nu citit din cod.

Ce s-a închis (verificat prin rulare)

	Runda 1	Acum
B1	bucla infinită lângă miezul nopții	✅ cursor `LocalDateTime`; testul cu `@Timeout(2)` trece
B2	`update` sărea peste program / time-off, `endTime` liber	✅ în service: `endTime` recalculat, aceleași validări ca la `add` (fără endpoint, vezi mai jos)
B3	orice eroare de business → 500	✅ `GlobalExceptionHandler`: procedură 99 → **404**, client 99 → **404**, `[1,1]` → **400**, `startTime` lipsă → **400** cu `details`
B4	suprapunerea nu era prinsă	✅ 10:15 peste 10:00–10:30 → **409**, aceeași programare de 2 ori → **409**, `availability` nu mai oferă 09:45, 10:00, 10:15
M1	nume de query inversate + duplicate	✅

	Runda 1	Acum
M2	64 de query-uri pe zi	✅ o interogare pe zi, comparat în memorie
M5	două excepții pentru același caz	✅
M6	pragul `24*60`	✅ scos
M7	seeder în orice profil, șterge programările	✅ `@Profile("dev")`, `deleteAll` comentat
M8	`contextLoads` fără MySQL	✅ profil `test` pe H2
M9	`procedures=[]` la update/confirm/...	✅

Nu mai e nicio 🔴 deschisă. Ai reparat regula, nu doar linia: la B3 ai prins toate excepțiile de business într-un singur loc, cu un test pentru fiecare (`GlobalExceptionHandlerTest`, 312 linii).

🟡 Importante

- **M1** O singură excepție pentru situații diferite → mesajul îl minte pe client. Codul HTTP e bun, textul nu:
  - programare în **trecut** (`2020-01-06T10:00`) → 400 *„Appointment end time must be after start time"* (`AppointmentCommandServiceImpl.java:84-85`, la fel la update `:198-199`);
  - **duminică** (frizerul nu lucrează) → 409 *„The selected time slot is already booked"* (`:334-336`);
  - **16:45 + 30 min**, iese din program → același 409 *„already booked"* (`:341-344`);
  - **time-off** → tot *„already booked"* (`:358-360`).

  Cine face UI-ul va scrie „ora e ocupată" când de fapt frizerul e liber azi. Fă câte o excepție cu mesajul ei: `AppointmentInPast` (400), `BarberNotWorking` (409, „Barber does not work at this time") pentru ieșirea din program și time-off.
- **M2** (restul din M3, runda 1) `AppointmentAvailabilityServiceImpl.java:46` — `add` refuză acum trecutul, dar `availability` dă în continuare sloturi pentru o dată trecută și pentru orele deja trecute de azi. Clientul vede un slot liber, iar când îl alege primește 400.
- **M3** (restul din M4, runda 1) `AppointmentAvailabilityServiceImpl.java:67-83` — `add` verifică `Barber.active` și `Procedure.active`, `availability` nu. O procedură dezactivată are sloturi „libere" pe care `add` le refuză. Aceeași regulă, verificată într-un loc și uitată în celălalt.

🟢 Observații

- **C1** rămâne: `root/root` și `show-sql: true` în `application.yaml` (până la Docker).
- **C2** rămâne: 4 × `System.out.println` în `DataSeeder`; data time-off-ului tot hardcodată (`DataSeeder.java:123`).

- **C3** `update`, `confirm`, `cancel`, `complete` sunt scrise și testate, dar nu au niciun endpoint: aplicația expune doar `POST /api/appointments` și `GET /api/appointments/availability`. Nu e greșeală, e ce urmează. Intră în planul proiectului tău final.

Q&A

1. De ce o programare în trecut primește mesajul despre „end time", deși ora de sfârșit e corectă? Ce trebuie să conțină o excepție ca mesajul să fie întotdeauna adevărat?
2. `add` și `availability` trebuie să răspundă la aceeași întrebare: „pot programa aici?". Unde ai pune regulile comune (trecut, activ, program) ca să nu le mai uiți în una dintre ele?
3. Clientul trimite `procedureIds: [99]`. Urmărește drumul: ce excepție se aruncă, cine o prinde și de unde vine `404`?

Runda 1 — 2026-10-23 — sha 9470047

Scop: `feat: add appointment availability and demo data seeder`. Poarta de compilare: compilează, 321 din 322 de teste trec (singura eroare e `BookingApplicationTests.contextLoads`, vezi M8). Cele trei 🔴 de mai jos au fost **reproduse prin rulare** (copie în afara repo-ului tău, pe H2), nu deduse din citire.

Tiparul rundei: regulile de business stau într-o singură metodă (`addAppointment`), iar celelalte drumuri (update, availability) le ocolesc sau le reimplementează diferit.

🔴 Critice

B1 🔴 Buclă infinită în availability când programul se termină aproape de miezul nopții

`AppointmentAvailabilityServiceImpl.java:101` (cauza reală: `:125`)

CUM E ACUM: `currentTime` e `LocalTime` și crește cu 15 minute (`:125`). Bucla se oprește doar când `date.atTime(currentTime)` + durata trece de `date.atTime(closingTime)` (`:101–103`). Cu un program care se termină la 23:59 și o procedură mai scurtă de 15 minute, cererea nu se mai termină niciodată.

Repro: H2 (MODE=MySQL), `data.sql` cu frizerul 100, procedura 100 „Quick Trim" de 10 min și programul de luni 22:00-23:59. Apoi `curl -m 5 'localhost:18082/api/appointments/availability?barberId=100&date=2026-10-26&procedureIds=100'` -> curl exit 28 (timeout), HTTP 000. `ps` -> 106.9% CPU, TIME 0:33 și crește. `jstack` -> `http-nio-18082-exec` stă în `AppointmentAvailabilityServiceImpl.getAvailableSlots(AppointmentAvailabilityServiceImpl.java:108)`.

DE CE: `LocalTime` e un ceas de perete: `23:45.plusMinutes(15)` dă `00:00`, nu „ziua următoare", pentru că `LocalTime` nu are dată. La pasul următor faci `date.atTime(00:00)`, adică 00:00 din **aceeași** zi. Asta e înainte de 23:59, deci condiția din `while` e din nou adevărată, bucla o ia de la miezul nopții și ajunge iar la 23:45, la nesfârșit.

De-aia curl a ieșit cu timeout, procesul a stat pe ~105% CPU, iar thread dump-ul arată thread-ul Tomcat blocat la linia 108, care face 2 query-uri la fiecare tură. Cât timp lucrezi cu `LocalTime`, nu vezi trecerea în ziua următoare. Ca s-o vezi, trebuie să numeri în `LocalDateTime`.

FIX: ține cursorul ca `LocalDateTime` : `slotStart = date.atTime(schedule.getStartTime())` , apoi `while (!slotStart.plusMinutes(totalDuration).isAfter(closingDateTime)) { ...; slotStart = slotStart.plusMinutes(15); }` , iar în listă pui `slotStart.toLocalTime()` .

- același bug / aceeași cauză la:
- `AppointmentAvailabilityServiceImpl.java:125` — `currentTime = currentTime.plusMinutes(15)` e cauza reală; linia 101 doar o expune. Fix-ul se face pe `:125` / `:96` , nu doar pe `:101` .
- `AppointmentAvailabilityServiceImpl.java:81` — garda `totalDuration >= 24*60` nu previne wrap-ul; bucla se blochează și la durate mici.
- Verificat: nu mai există alt `LocalTime` folosit ca variabilă de buclă. Celelalte `plusMinutes` din `src/main` sunt pe `LocalDateTime` (`AppointmentAvailabilityServiceImpl.java:102` și `:106` , `AppointmentCommandServiceImpl.java:113`), deci nu fac wrap. Singurul alt `while` din `booking-api/src/main` e cel de la `:101` ; în `booking-client` și `booking-mobile` nu sunt bucle de timp.
- Verificat, NU e încălcare: `AppointmentCommandServiceImpl.java:295-297` folosește `toLocalTime()` pe `end` , dar e protejat de `startTime.toLocalDate().equals(endTime.toLocalDate())` pe aceeași linie. `DataSeeder.java:105-106` (09:00-17:00) nu declanșează bug-ul.
- Test: nu există niciunul pentru un program care se termină la 23:59 — adaugă unul, e exact cel care ar fi prins bug-ul.

B2 🔴 `updateAppointment` sare peste program și peste time-off, iar `endTime` e liber

`AppointmentCommandServiceImpl.java:194`

CUM E ACUM: `addAppointment` validează intervalul, programul frizerului, time-off-ul și suprapunerea (`:115-128`). `updateAppointment` validează doar intervalul și suprapunerea (`:194-213`). Pe deasupra, `endTime` vine din cerere (`:188`) și nu mai e calculat din durata procedurilor.

Repro (test `@SpringBootTest` în copie, H2, datele din `DataSeeder` : luni 09-17, pauză 12-13 pe 2026-10-26): - `addAppointment(barber 1, 2026-10-26T12:00, [1])` -> `AppointmentTimeSlotUnavailable` (corect). - `addAppointment(10:00)` -> id=1, 10:00-10:30. - `updateAppointment(1, 12:00, 12:30)` -> OK, deși e pauza de masă; răspunsul are `procedures=[]` . - `updateAppointment(1, 2026-11-01T03:00, 03:05)` -> OK, duminică 03:00-03:05, `status=BOOKED` , deși frizerul nu lucrează duminica, iar tunsoarea de 30 de minute a ajuns la 5.

DE CE: regulile de business stau în metoda de creare, nu în modelul programării. Orice drum care schimbă orele fără să treacă prin `addAppointment` ocolește deci regulile. Entitatea `Appointment` acceptă orice pereche start/end, iar baza de date nu știe nimic de program, așa că nicio plasă de dedesubt nu prinde greșeala.

Asta ai văzut în rulare: aceeași programare pică la 12:00 prin `add` și trece prin `update` . Tot de-aia răspunsul vine cu `procedures=[]` : update-ul nu mai ține legătura cu procedurile, deci nici durata nu mai are de unde să vină.

FIX: în `updateAppointment` calculează `endTime = startTime + suma durationMinutes` din `appointmentProcedures` , scoate `endTime` din `AppointmentUpdateRequest` și cheamă aceleași `validateBarberSchedule` + `validateBarberTimeOff` ca la add. Cel mai simplu: o singură metodă privată `validateSlot(barberId, start, end, excludeId)` , folosită și de add, și de update.

- aceeași regulă încălcată / de ținut cont la:
- `AppointmentCommandServiceImpl.java:194-196` — după `validateTimeInterval` lipsesc `validateBarberSchedule` și `validateBarberTimeOff` ; trebuie apelate pe start/end rezultate și pentru

update parțial (doar `startTime` sau doar `endTime`). Verificarea de „aceeași zi" din `validateBarberSchedule` (`:297–302`) respinge și intervalele pe mai multe zile.

- `AppointmentCommandServiceImpl.java:162–171` — blocul `startTime`/`endTime` din update: `endTime` acceptat liber, fără recalculare din durata procedurilor (compară cu `:114` din `addAppointment`).
- `AppointmentCommandServiceImpl.java:197–212` — verificarea de suprapunere rulează doar dacă `timeChanged`; programul și time-off-ul nu sunt verificate nicăieri în update. Mută validarea completă în acel bloc `timeChanged`.
- `AppointmentCommandServiceImpl.java:224` — `return AppointmentResponse.from(appointment)` dă `procedures=[]` (`AppointmentResponse.java:55–57`). Defect conex, nu aceeași regulă (vezi M9).
- Nicio altă cale nu mai scrie `startTime`/`endTime`: `setStartTime`/`setEndTime` apar doar la `:215–216`. `AppointmentAvailabilityServiceImpl.java:88–125` verifică program, programări suprapuse și time-off, deci respectă regula.
- Pentru mai târziu (nu e încălcare acum): în `src/main` nu există serviciu sau controller care să creeze programări din `RecurringAppointment`, iar `BarberSchedule`/`BarberTimeOff` sunt create doar în `DataSeeder.java:100–120`. Când adaugi generarea de programări recurente sau endpoint-uri pentru program/time-off, ele trebuie să treacă prin aceleași validări.
- Teste: grupul de update din `AppointmentCommandServiceImplTest` nu acoperă program/time-off la update. Adaugă teste pentru pauză, zi liberă, depășirea orei de închidere și interval pe mai multe zile.

B3 🔴 Orice eroare de business iese ca 500 Internal Server Error

`AppointmentAvailabilityController.java:24`

CUM E ACUM: în proiect nu există niciun `@RestControllerAdvice` și niciun `@ResponseStatus`, iar toate excepțiile tale extind `RuntimeException`. O procedură inexistentă, o procedură dublată, o listă goală sau un frizer greșit dau toate 500, fără mesaj în body.

Repro (`java -jar` pe H2, cu seeder-ul): `GET /api/appointments/availability?barberId=1&date=2026-10-26&procedureIds=99` -> `500 {"status":500,"error":"Internal Server Error"}`; `procedureIds=1,1` -> 500 (în log: `AppointmentDuplicateProcedure`); `barberId=2&procedureIds=1` -> 500; `procedureIds=` -> 500 (`AppointmentProceduresRequired`). Controlul: `procedureIds=1,2` -> 200, 09:00..11:15 și 13:00..16:15 (pauza 12-13 exclusă corect).

DE CE: Spring transformă o excepție în cod HTTP numai dacă cineva îi spune cum: un `@ExceptionHandler`, un `@ResponseStatus` pe clasă sau `ResponseStatusException`. O excepție necunoscută ajunge la `DispatcherServlet` ca „Request processing failed". Tomcat o tratează ca defect al serverului, o loghează cu stack trace (vezi `ERROR` în log) și face forward pe `/error`, care întoarce 500 cu body-ul generic.

Mesajul tău frumos („One or more selected procedures were not found") apare doar în logul serverului. Clientul nu-l vede și nu are cum să deosebească „ai greșit tu cererea" de „a căzut serverul".

FIX: un `@RestControllerAdvice` cu `@ExceptionHandler` pe grupuri: `*NotFound` -> 404, `*Duplicate*`/`*Required`/`*InvalidTimeInterval` -> 400, `*TimeSlotUnavailable`/`*InvalidStatus`/`*CannotBeCanceled` -> 409. Body-ul: `ProblemDetail` cu `getMessage()`. Același handler acoperă și excepțiile din barbers/customers/procedures/users, chiar dacă încă nu au controller. Contractul tău din `booking-docs/openapi.yaml:8` promite deja `ProblemDetail` (RFC 9457), dar nu e implementat.

- aceeași cauză (excepții care vor ieși 500) la:

- `AppointmentAvailabilityServiceImpl.java:48` (`AppointmentProceduresRequired` -> 400), `:53` (`AppointmentProcedureNotFound` pentru id null -> 400), `:60` (`AppointmentDuplicateProcedure` -> 400/409), `:66` și `:71` (`AppointmentProcedureNotFound` -> 404). Același endpoint nu validează `barberId` inexistent (nu există `BarberNotFound` aici).
- `AppointmentCommandServiceImpl.java`: `:69` `CustomerNotFound`, `:72` `BarberNotFound`, `:75` și `:280` `AppointmentInvalidTimeInterval`, `:79` `AppointmentProceduresRequired`, `:84` / `:99` `AppointmentProcedureNotFound`, `:91` `AppointmentDuplicateProcedure`, `:107` `AppointmentProcedureBarberMismatch`, `:128` / `:211` / `:293` / `:299` / `:315` `AppointmentTimeSlotUnavailable` (-> 409), `:174` / `:232` / `:263` `AppointmentInvalidStatus`, `:248` `AppointmentCannotBeCanceled`, `:275` `AppointmentNotFound`.
- `AppointmentQueryServiceImpl.java:35` (`AppointmentNotFound`).
- `CustomerCommandServiceImpl.java`: `:38` `UserNotFound`, `:41` `CustomerUserAlreadyExists`, `:45` și `:79` `CustomerPhoneAlreadyExists`, `:65` și `:97` `CustomerNotFound`; `CustomerQueryServiceImpl.java:29`.
- `ProcedureCommandServiceImpl.java`: `:37` `BarberNotFound`, `:41` și `:69` `ProcedureNameAlreadyExists`, `:62` și `:99` `ProcedureNotFound`; `ProcedureQueryServiceImpl.java:29`.
- `BarberCommandServiceImpl.java`: `:37` `UserNotFound`, `:40` `BarberUserAlreadyExists`, `:60` și `:86` `BarberNotFound`; `BarberQueryServiceImpl.java:29`.
- `BarberSchedule.java:54` (`BarberScheduleStartTimeMustBeBeforeEndTime`) și `:65` (`IllegalArgumentException` în `updateWorkingHours`, mesaj în engleză, tot 500).
- `UserCommandServiceImpl.java`: `:31` și `:53` `UserEmailAlreadyExists`, `:48` / `:71` / `:85` `UserNotFound`; `UserQueryServiceImpl.java:28`.
- Cauza comună: lipsa totală a unui `@RestControllerAdvice` în `booking-api/src/main`. Un singur handler le rezolvă pe toate; fără el, fiecare endpoint nou repetă problema.

🟡 Importante

- **M1** `AppointmentRepository.java:33` și `:58` — numele sunt inversate: `hasOverlappingAppointmentExcludingId` NU exclude niciun id, iar `hasOverlappingAppointment` exclude. Plus `:41` și `:67`: aceleași interogări scrise ca derived queries, nefolosite. Păstrează câte una din fiecare, cu nume corecte.
- **M2** `AppointmentAvailabilityServiceImpl.java:108-119` — 2 query-uri pe fiecare slot, deci 64 de query-uri pentru o zi 09-17. la o dată programările și time-off-ul zilei (`findByBarberIdAndStartTimeBetween`) și compară în memorie.
- **M3** `AppointmentAvailabilityServiceImpl.java:42` — dă sloturi și pentru o dată din trecut (`date=2020-01-06` -> 200 cu 31 de sloturi) și pentru orele deja trecute de azi. `addAppointment` nu verifică nici el că `startTime` e în viitor.
- **M4** `AppointmentAvailabilityServiceImpl.java:69` și `AppointmentCommandServiceImpl.java:104` — `Procedure.active` și `Barber.active` nu sunt verificate, deci o procedură dezactivată se poate programa.
- **M5** `AppointmentAvailabilityServiceImpl.java:71` — o procedură a altui frizer aruncă `AppointmentProcedureNotFound`, iar `add` aruncă `AppointmentProcedureBarberMismatch`. Același caz, două excepții.
- **M6** `AppointmentAvailabilityServiceImpl.java:81` — pragul de `24*60` nu are sens (schedule-ul limitează deja), poate fi scos; nu protejează de B1.

- **M7** `DataSeeder.java:25` — rulează în orice profil (importul `Profile` de la `:17` e nefolosit); data time-off-ului e hardcodată la 2026-10-26 (`:111`). Pune `@Profile("dev")` .
- **M8** `BookingApplicationTests.contextLoads` pică fără MySQL pornit local („Unable to determine Dialect"): 321/322 trec, 1 eroare. Dă-i un profil de test pe H2 (h2 e deja în `pom`).
- **M9** `AppointmentCommandServiceImpl.java:222` — update/confirm/cancel/complete întorc `procedures=[]` , fiindcă `AppointmentResponse.from(appointment)` nu încarcă procedurile.

🟢 Cleanups

- **C1** `application.yaml` : `root/root` și `show-sql: true` . Pentru dev e ok; mută-le în variabile de mediu când ajungi la Docker.
- **C2** `System.out.println` în `DataSeeder` : folosește un logger.

Ce e bine făcut

- Compilează, iar 321 din 322 de teste trec (singura eroare e `contextLoads` , din cauza MySQL).
- Interogarea de suprapunere cu `<` și `>` stricte e corectă: programările lipite una de alta trec (11:15-12:00 apare, 11:30 nu, pentru că se lovește de pauză).
- Validările pe `procedureIds` (null, duplicate, inexistente, alt frizer) sunt făcute înainte de orice calcul.
- `AppointmentProcedure` păstrează prețul și durata din momentul programării. Bun model.
- Invariantul start < end e în constructorul `BarberSchedule` , iar `UniqueConstraint` e pus pe `(barber_id, day_of_week)` .
- `open-in-view: false` , interfețe Command/Query separate.

Before / After (🔴)

B1 — cursorul în `LocalDateTime`

Before	After
`LocalTime currentTime = schedule.getStartTime();`	`LocalDateTime slotStart = date.atTime(schedule.getStartTime());`
`while (!date.atTime(currentTime).plusMinutes(totalDuration).isAfter(date.atTime(closingTime))) {`	`while (!slotStart.plusMinutes(totalDuration).isAfter(closingDateTime)) {`
`currentTime = currentTime.plusMinutes(15);` (23:45 -> 00:00 în aceeași zi)	`slotStart = slotStart.plusMinutes(15);` (23:45 -> 00:00 a doua zi, depășește `closingDateTime`)
lista primește `currentTime`	lista primește `slotStart.toLocalTime()`

B2 — o singură validare, folosită de add și de update

Before	After
`updateAppointment` : `validateTimeInterval (start, end)` + suprapunere; `endTime` din request	`updateAppointment` : `endTime = startTime + suma durationMinutes` , apoi `validateSlot (barberId, start, end, appointmentId)`
`addAppointment` : schedule + time-off + suprapunere, scrise inline	`addAppointment` : același `validateSlot(barberId, start, end, null)`
`AppointmentUpdateRequest` are `endTime`	`AppointmentUpdateRequest` fără `endTime`

B3 — un singur loc care transformă excepția în HTTP

Before	After
`throw new AppointmentProcedureNotFound()` -> 500 `{"status":500,"error":"Internal Server Error"}`	același `throw` , dar un `@RestControllerAdvice` cu `@ExceptionHandler(AppointmentProcedureNotFound.class)` -> 404
mesajul apare doar în logul serverului	body `ProblemDetail` cu `getMessage()` , clientul îl citește
`AppointmentTimeSlotUnavailable` -> 500	`AppointmentTimeSlotUnavailable` -> 409

Q&A

1. De ce `23:45` + 15 minute dă `00:00` în `LocalTime` , dar `00:00` de a doua zi în `LocalDateTime` ? Ce informație îi lipsește `LocalTime` ca să știe că s-a schimbat ziua?
2. Ai pus validarea programului și a pauzei în `addAppointment` . Unde ar trebui să stea regula „o programare e doar în orele de lucru, fără time-off", ca să nu o poată ocoli update-ul sau o metodă nouă de reprogramare?
3. Când serviciul aruncă `AppointmentProcedureNotFound` , cine decide că răspunsul e 500? Ce trebuie să-i spui lui Spring ca să devină 404, iar un client să poată citi mesajul tău?